

Week 01 Explained Student Worksheet

Python File Manipulation for Information Retrieval

Name _____ Student ID _____ Group _____

Create a small collection of text files and load it into a Python dictionary. This collection is the input to the retrieval programs developed in later weeks.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Create a docs directory containing three UTF-8 text files.
- Read, write, and append text using context managers.
- Load several files into a dictionary keyed by document ID.
- Compare `read()`, `readline()`, and `readlines()` using the same file.

Understand the concepts

A document is one unit that a retrieval system can return, such as a text file. A collection is the set of documents being searched. In this lab, a filename without `.txt` becomes a document ID, and the dictionary stores each ID with its text.

A relative path is interpreted from the current working directory. The supplied program first finds the directory containing its own script, then joins `docs` or `data` to that directory. This lets the program run from a terminal opened elsewhere. `os.path.join` builds paths using the correct Windows separator.

Read mode `r` opens an existing file without changing it. Write mode `w` creates a file or replaces its contents. Append mode `a` adds text to the end. A `with` block closes the file even when an error occurs. Specify UTF-8 when creating and reading the collection.

`read()` returns the remaining text as one string. `readline()` reads one line. `readlines()` returns the remaining lines as a list. These operations advance the file position. Reopen the file to start reading from the beginning.

Predict before running

Before running, explain how `doc1.txt` becomes a dictionary key and what its corresponding value should contain.

Prepare your terminal

Extract the course and open a terminal inside Week_01_File_Manipulation. Keep docs and data beside the Python files. The main demonstration is lab01_file_manipulation.py; your exercise starter is exercises_lab01.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab01_file_manipulation.py
py -3.13 exercises_lab01.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Run this complete example inside a disposable practice directory. The first block creates a file; the second reads it. The shorter form requested in the course material works with the platform default encoding; the full laboratory program adds encoding="utf-8" for predictable text handling.

Use encoding="utf-8" when creating your actual collection. The small example below uses the platform default encoding, matching the original lab illustration.

```
with open("doc1.txt", "w") as f:
    f.write("Python helps us process text.\n")
with open("doc1.txt", "r") as f:
    text = f.read()
print(text.strip())
```

Expected output from an executed example

Python helps us process text.

How each stage produces the result

1. The first with block opens a practice file in write mode. A file with that name is created if absent; an existing file is emptied. Run this example in a disposable practice directory, not in your collection.
2. write sends the string to the open file. The newline character ends the line. Leaving the with block closes the file, so the second block can reopen it for reading.
3. read returns the whole remaining file as a string. The variable text survives after the file closes. strip removes surrounding whitespace only from the displayed string.
4. In the full program, readline advances past the first line and readlines returns only what remains. Reopening notes.txt resets the reading position. Existing docs files are preserved, but notes.txt is deliberately reset on every run.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab01_file_manipulation.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def load_documents(folder_path):
    documents = {}
    for filename in sorted(os.listdir(folder_path)):
        path = os.path.join(folder_path, filename)
        if filename.endswith(".txt") and os.path.isfile(path):
            with open(path, "r", encoding="utf-8") as file:
                documents[filename[:-4]] = file.read()
    return documents
```

Code walkthrough

1. documents starts as an empty dictionary. sorted(os.listdir(folder_path)) gives a repeatable filename order; os.path.join constructs each complete path.
2. The suffix check accepts lowercase .txt names. isfile excludes directories, even if a directory happens to have a name ending in .txt.
3. The with block reads UTF-8 text. filename[:-4] removes the last four characters, .txt, to create the document ID. The dictionary retains the full text, including newline characters.
4. BASE_DIR in the main program is derived from __file__, not the terminal location. The shipped data directory must remain beside the script; the program creates docs if needed but does not recreate a missing data directory.

After two readline calls on the two-line notes file, what should read return? Explain the file position.

Connection to the course

The next labs need a dictionary of document IDs and text. Correct loading keeps later differences in search results attributable to algorithms rather than missing or overwritten data.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Open the Week_01_File_Manipulation folder in your editor. Identify data/welcome.txt and the two Python files.

Identify which files are input data and which are executable programs. Do not open the documents in write mode merely to inspect them.

Step 2 Predict what each file-reading method will return. Run the solution once to create docs and the demonstration notes file.

Write your predicted return types first: a string for read and readline, a list for readlines. Use repr to make newline characters visible.

Step 3 Inspect docs/doc1.txt, doc2.txt, and doc3.txt. Record each document ID and its text.

Record the mapping between filenames, document IDs, and contents. IDs are names rather than numeric positions in a list.

Step 4 Trace the sorted directory loop. Explain why both the .txt suffix and isfile check are used.

Follow one filename through path construction, filtering, reading, and dictionary insertion. Add a harmless non-text file to test the filter.

Step 5 Modify a document, rerun the solution, and confirm that your edit is preserved.

Change a sentence in doc1.txt, save it, and compare the next output. This checks that the existence guard protects existing collection files.

Step 6 Complete the starter loader and print every dictionary key and value. Run it from a different working directory.

Implement the loader yourself. Run the file by absolute path from another directory to verify that script-relative paths still find docs.

Test record

Check to demonstrate	Actual observation	Pass or revise
Three original documents load with the correct IDs		
A fourth .txt file loads but a .csv file is skipped		
A saved document edit survives a second run		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: add doc4.txt about databases and confirm that the loader finds it.
- Implementation: finish load_documents in exercises_lab01.py using with open and UTF-8.
- Investigation: call readline() twice and then read(). Predict the three results before running.
- Debugging: explain why opening an existing document in w mode before reading loses its earlier text. Reproduce this only in a disposable file.

Diagnose common errors

Old text disappears. Opening with w truncates an existing file immediately. Use r to inspect it and a only when you intend to add text.

A file cannot be found. Check extraction and folder placement. The supplied program uses its own location, while a short open("file.txt") example uses the terminal working directory.

Submit these items

- Your completed exercise file and a docs folder with at least four short documents.
- A terminal transcript showing all document IDs and contents, plus one run from another directory.
- A short explanation of the difference between the three read methods and the effects of w and a.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

What is the difference between a document and a collection?

Which directory supplies the starting point for an ordinary relative path?

Optional challenge

Write a separate collection summary file containing one line per document: ID, character count, and line count. Keep it outside docs so it does not become a searchable document.

Source for this companion

Based on the supplied Week 01 lab sheet, shipped program, exercise starter, and sample data. The complete main program and expected_output.txt in this folder are the references for checking your baseline run.